

University of Applied Sciences Hamburg	Bus Systems	 Hochschule für Angewandte Wissenschaften Hamburg Hamburg University of Applied Sciences
Group No: 1	Lab 4 Room 8.65	Efe Kardes Matriculation No: 2757072
Date: 08/07/2026		Emir Avni Sahin Matriculation No: 2756541
Professor: Ratmus Rettig		Stanislaw Chilecki Matriculation No: 2761057

Report History

Report 1		Date		Remarks
	received			
	checked			
	result	o.k.		
		n.g.		

Prof.

Contents

Short Introduction: What is CAN?	2
1 Part 1: CAN Interface Setup and Parameter Testing	2
1.1 Task in Our Words	2
1.2 Setup	3
1.3 Expected Values	5
1.4 Collected Data	5
1.5 Analysis	6
2 Part 2: Physical Layer and Oscilloscope Measurements	6
2.1 Task in Our Words	6
2.2 Setup and Expected Values	7
2.3 Collected Oscilloscope Data	9
2.4 Measurement Summary	10
2.5 Analysis	10
3 Part 3: Sensor Data Transfer over CAN	11
3.1 Task in Our Words	11
3.2 Structure and Hardware Data Flow	11
3.3 Payload Design	13
3.4 Collected Data	14
3.5 Analysis	14
4 Part 4: Automatic Error Checking	14
4.1 Task in Our Words	14
4.2 Automatic Checks Implemented	15
4.3 Successful 100-message Test	15
4.4 Additional Detection Test with Sensor Errors	15
4.5 Analysis	16
5 Part 5: Setup of a Large CAN Network	17
5.1 Task in Our Words	17
5.2 Network Setup and Hardware Precautions	17
5.3 Collected Data & External Receive Proof	17
5.4 Analysis	18
6 Conclusion	18
A Key Source Code Extracts	20
A.1 Data Payload Encoding and Decoding (Part 3)	20
A.2 Automatic Error Verification Logic (Part 4)	20
A.3 Shared Bus Non-Blocking Loop (Part 5)	21

Short Introduction: What is CAN?

CAN means Controller Area Network. It is a serial communication bus used when several electronic controllers must exchange short messages reliably. A CAN bus uses two signal lines, `CAN_H` and `CAN_L`. The useful signal is the voltage difference between both lines. This makes CAN robust against external noise because common-mode disturbances appear on both lines and mostly cancel out in the differential receiver.

In this lab the bus was first tested as a local loopback between two channels of the USB-CAN interface. Later, a 1-wire temperature sensor was read by an Arduino and the temperature value was encoded into an 8-byte CAN payload. At the end, the interface was connected to a common laboratory CAN bus.

1 Part 1: CAN Interface Setup and Parameter Testing

1.1 Task in Our Words

The first task was to install and test the USB-CAN interface. CAN channel 1 and CAN channel 2 were connected in loopback mode by wiring `CAN1_H` to `CAN2_H` and `CAN1_L` to `CAN2_L`. To satisfy strict physical layer requirements and avoid signal reflection, termination resistors ($120\,\Omega$) were activated for this local two-node setup. Furthermore, to explicitly prevent floating connections, 5V VCC and Ground were natively supplied via the USB connection to the host PC, ensuring a common reference voltage for both transceivers.

1.2 Setup

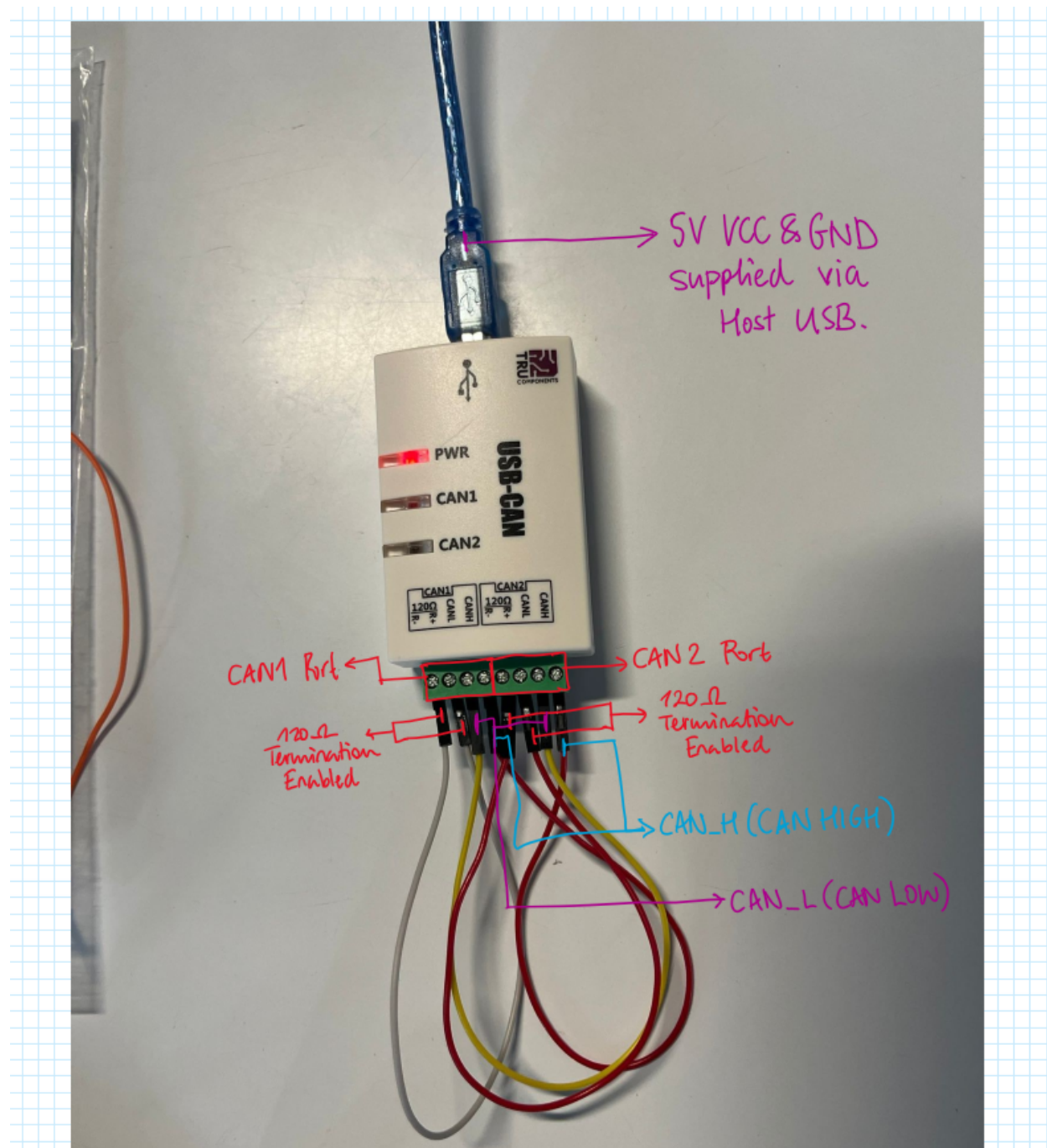


Figure 1: USB-CAN loopback overview.

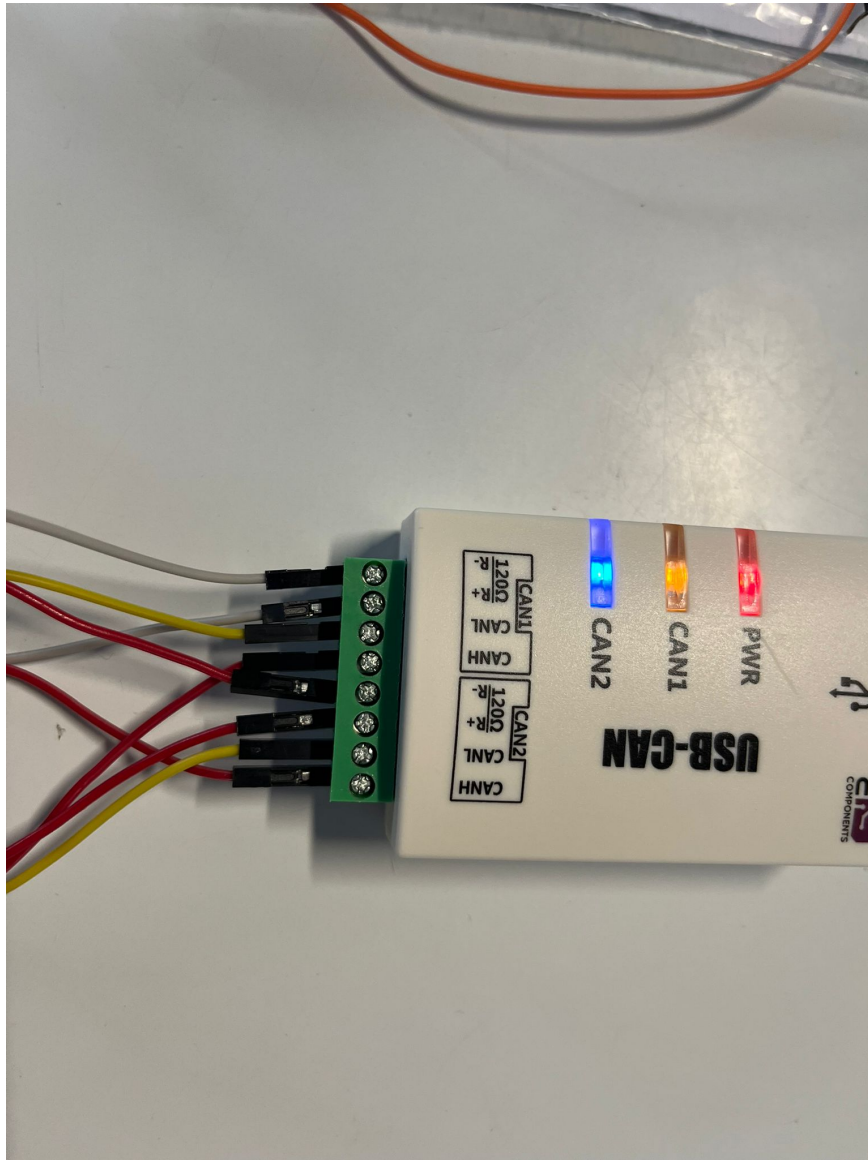


Figure 2: Physical USB-CAN setup used for the local CAN1–CAN2 loopback test. The photos show the TRU-Components USB-CAN adapter, jumper wiring, and the active interface LEDs.

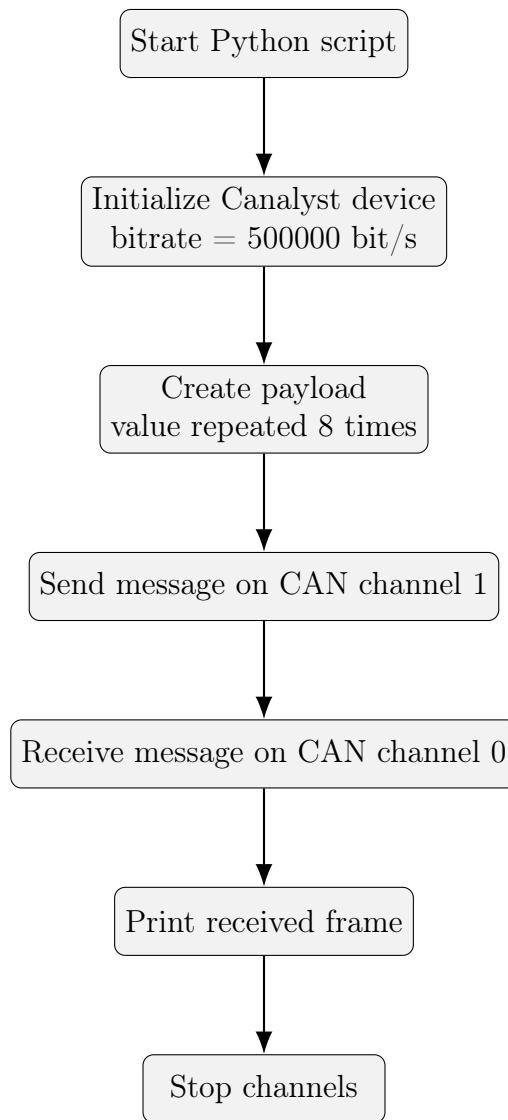


Figure 3: Program flow for the basic CAN loopback test.

1.3 Expected Values

The bitrate must be identical for both CAN nodes. If both channels are initialized with the same bitrate, valid standard CAN frames should be transmitted and received. The identifier, DLC and payload can be changed per frame as long as the selected values are valid. Lower CAN identifiers have higher arbitration priority on a shared CAN bus.

1.4 Collected Data

The baseline test used CAN ID 0x3FE, standard frame format, `remote=False`, `extended=False`, DLC = 8, and an iterating payload from 00 to FF. Example terminal output:

```

1 CanalytMessage ID=0x3fe TS=0x3878f7 Data=0000000000000000
2 CanalytMessage ID=0x3fe TS=0x387cef Data=0101010101010101
3 CanalytMessage ID=0x3fe TS=0x3880f2 Data=0202020202020202
4 ...
5 CanalytMessage ID=0x3fe TS=0x38be7a Data=1111111111111111
6 ...
7 CanalytMessage ID=0x3fe TS=... Data=ffffffffffffffff

```

Listing 1: Baseline loopback terminal output.

The second test changed the CAN ID to 0x100 and used a fixed payload.

```
1 CanalystMessage ID=0x100 TS=0x8e8e46 Data=1122334400000000
```

Listing 2: Static payload with modified identifier.

A further test changed the bitrate to 250 kbit s^{-1} . Communication still worked after the device was re-initialized with the same bitrate for both channels.

Table 1: Part 1 parameter tests.

Test	Bitrate	CAN ID	Payload / DLC	Result
Baseline	500 kbit/s	0x3FE	Iterating 8-byte array (0x00 to 0xFF)	Received correctly
ID change	500 kbit/s	0x100	11 22 33 44 00 00 00 00	Received correctly
DLC test	500 kbit/s	0x100	DLC configured to 4	Frame accepted; printout shows 8-byte buffer
Bitrate test	250 kbit/s	0x100	Same payload	Received correctly after reinitialization

1.5 Analysis

The test confirmed that the two-channel loopback setup worked. Changing the CAN identifier changed the ID shown in the receiver output without disturbing the transmission. Reducing the bitrate to 250 kbit s^{-1} was successful because both channels used the same bitrate. The test confirmed that the two-channel loopback setup worked. Changing the CAN identifier changed the ID shown in the receiver output without disturbing the transmission. Reducing the bitrate to 250 kbit s^{-1} was successful because both channels used the same bitrate.

The DLC test reveals a critical hardware-software distinction. Physically, the CAN hardware strictly followed the `data_len=4` command and only transmitted 4 bytes across the differential bus before appending the CRC field. However, the `canalystii.Message` Python class maps to a fixed 8-byte C-types structure in memory. When Python's `print()` function is called, it naively dumps the entire 8-byte memory buffer to the console, padding the unused space with zeroes.

2 Part 2: Physical Layer and Oscilloscope Measurements

2.1 Task in Our Words

The second task was to measure the physical CAN signals with an oscilloscope. The objective was to observe `CAN_H`, `CAN_L` and the differential signal, compare 500 kbit s^{-1} with 250 kbit s^{-1} , and inspect special cases such as a zero payload where bit stuffing becomes visible.

2.2 Setup and Expected Values

Channel 1 was connected to **CAN_H** (confirmed retroactively by its positive deflection during dominant states) and Channel 2 to **CAN_L** (confirmed by its negative deflection). Crucially, both oscilloscope probe ground clips were connected directly to the oscilloscope's own chassis ground lug. This ensured a stable reference for the voltage measurements without creating a ground loop with the USB host.

The oscilloscope math channel represented the differential voltage:

$$V_{diff} = V_{CAN_H} - V_{CAN_L}$$

Expected values:

- Recessive state: **CAN_H** and **CAN_L** close to about 2.5 V; differential voltage close to 0 V.
- Dominant state: **CAN_H** rises and **CAN_L** falls; differential voltage roughly 2 V.
- At 500 kbit s⁻¹: expected bit time $T_b = 1/500000 = 2 \mu\text{s}$.
- At 250 kbit s⁻¹: expected bit time $T_b = 1/250000 = 4 \mu\text{s}$.

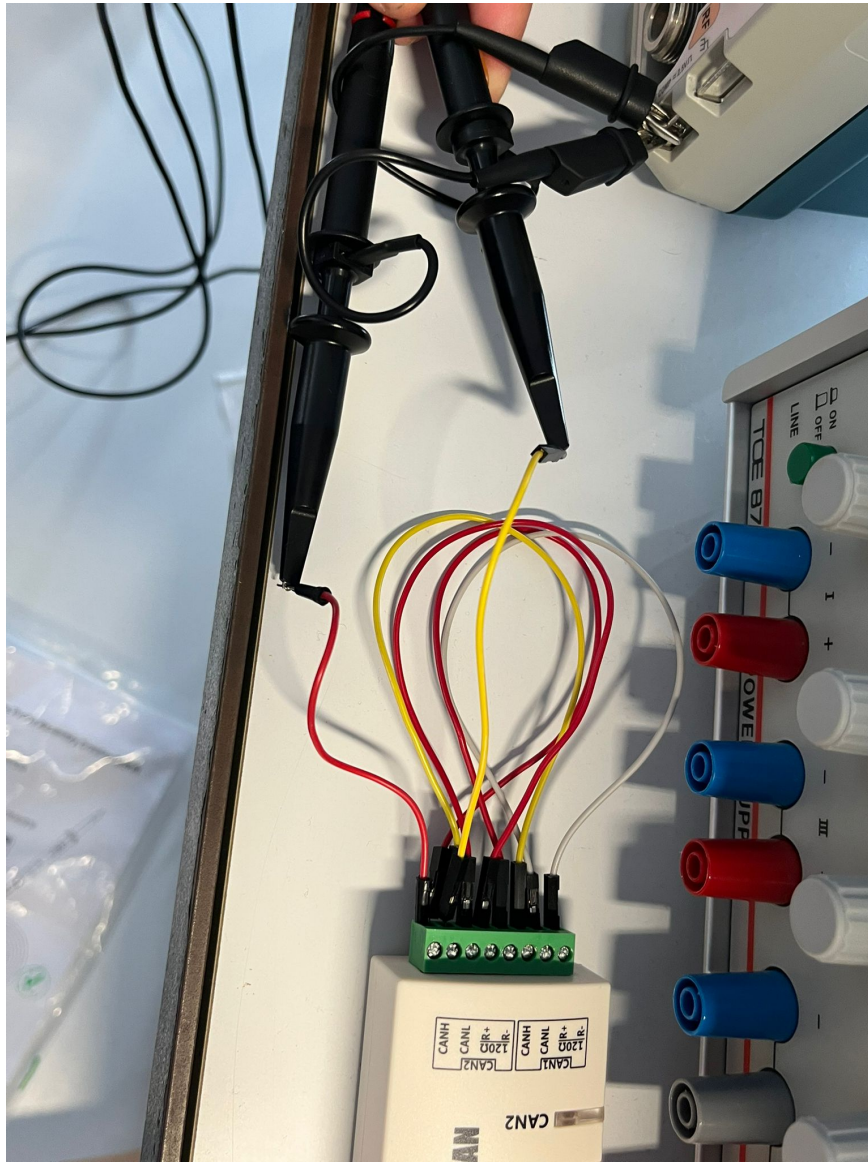


Figure 4: Oscilloscope and USB-CAN measurement setup (CAN_H connected to CH1, CAN_L connected to CH2)

2.3 Collected Oscilloscope Data

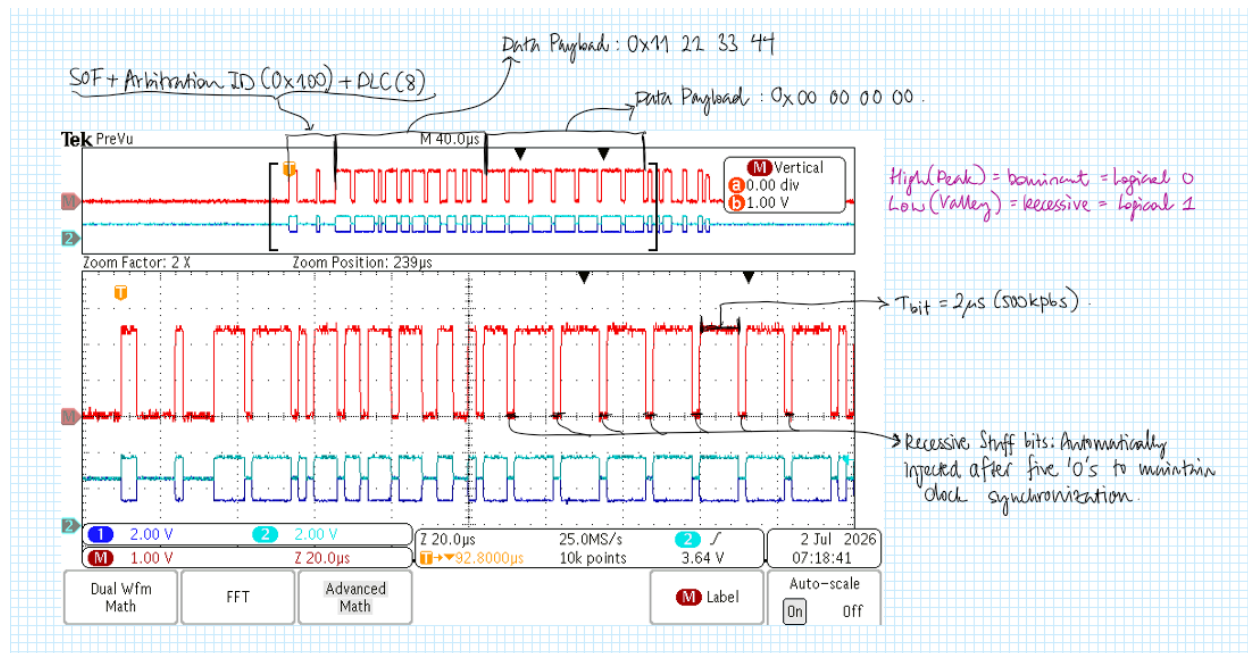


Figure 5: Zoomed CAN telegram at 500 kbit s^{-1} with payload 11 22 33 44 00 00 00 00.

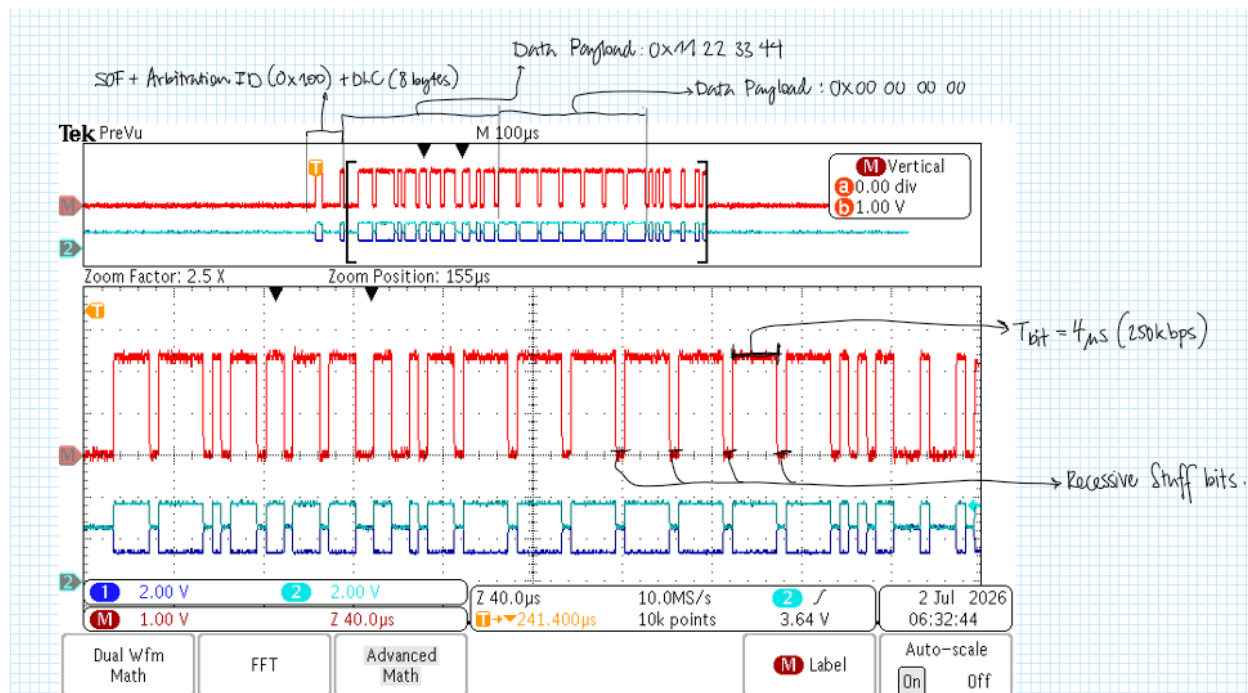


Figure 6: Zoomed CAN telegram at 250 kbit s^{-1} . The bit time is doubled compared with the 500 kbit s^{-1} measurement.

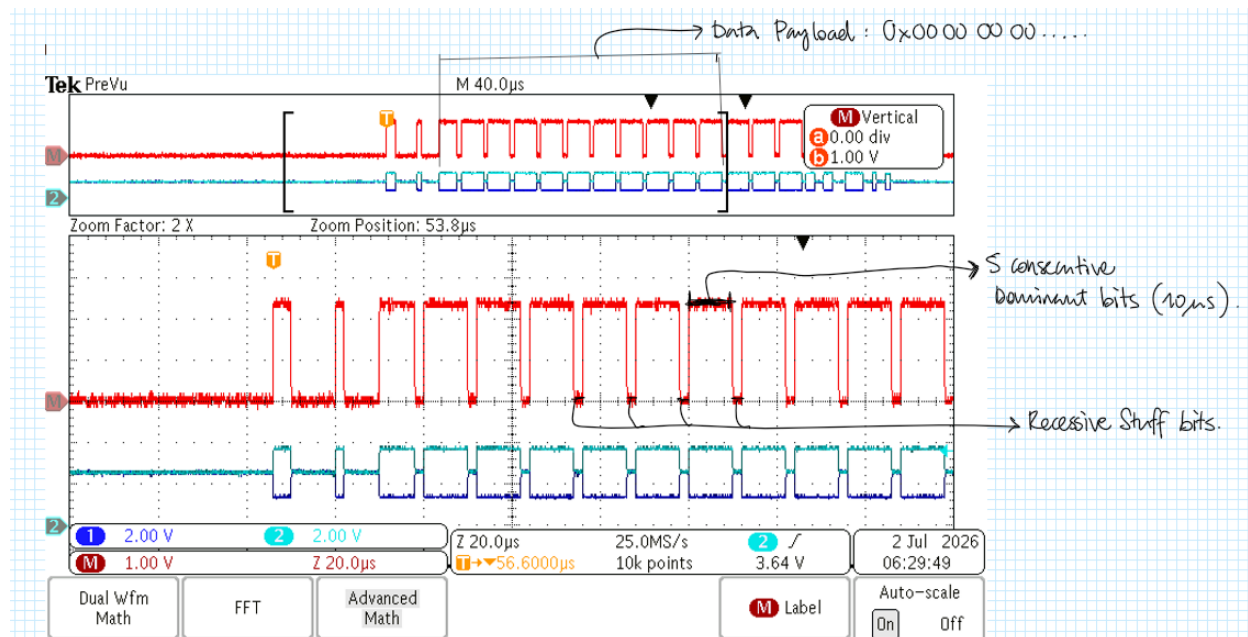


Figure 7: Zero-payload telegram at 500 kbit s^{-1} . The periodic transitions show the effect of CAN bit stuffing.

2.4 Measurement Summary

Table 2: Part 2 timing comparison.

Measurement	Expected bit time	Observed behavior	Comment
500 kbit/s	2 μs	Approx. 2 μs per bit	Normal frame timing
250 kbit/s	4 μs	Approx. 4 μs per bit	Frame is stretched in time
Zero payload	2 μs base bit time	Recessive stuff bits appear	Synchronization by bit stuffing

2.5 Analysis

The oscilloscope measurements confirmed the differential operation of the CAN physical layer. The fundamental advantage of this is noise immunity: if electromagnetic interference (EMI) strikes the bus, it induces an identical voltage spike on both tightly twisted wires (common-mode noise). Because the transceiver—and our Math channel—evaluates only the difference, $V_{\text{noise}} - V_{\text{noise}} = 0$. The noise mathematically cancels itself out.

Changing the bitrate changed the timing but not the physical voltage level. At 250 kbit s^{-1} , one bit lasted about twice as long ($4 \mu\text{s}$) as at 500 kbit s^{-1} ($2 \mu\text{s}$).

The zero-payload measurement explicitly demonstrated the protocol's clock synchronization mechanic. A payload of all zeroes constitutes 64 consecutive dominant states. Without transitions, receiving nodes would experience clock drift and misread the frame. The CAN hardware actively counts dominant bits; upon reaching five consecutive '0's, it deliberately injects a recessive '1' stuff bit. This forces a physical voltage edge, allowing receiving nodes to realign their internal clocks before automatically discarding the stuff bit.

3 Part 3: Sensor Data Transfer over CAN

3.1 Task in Our Words

The third task connected the 1-wire temperature sensor setup to the CAN bus test. A temperature value was read through the Arduino serial connection, encoded into an 8-byte CAN payload, transmitted on one CAN channel, received on the other channel, decoded and compared with the original value.

3.2 Structure and Hardware Data Flow

To explicitly verify the physical power and communication layers, Figure 8 demonstrates the physical connections, ensuring no elements were left floating. Following this, Figure 9 outlines the programmatic data conversions.



Figure 8: Explicit Hardware Block Diagram detailing the physical connections and power supplies for the sensor integration.

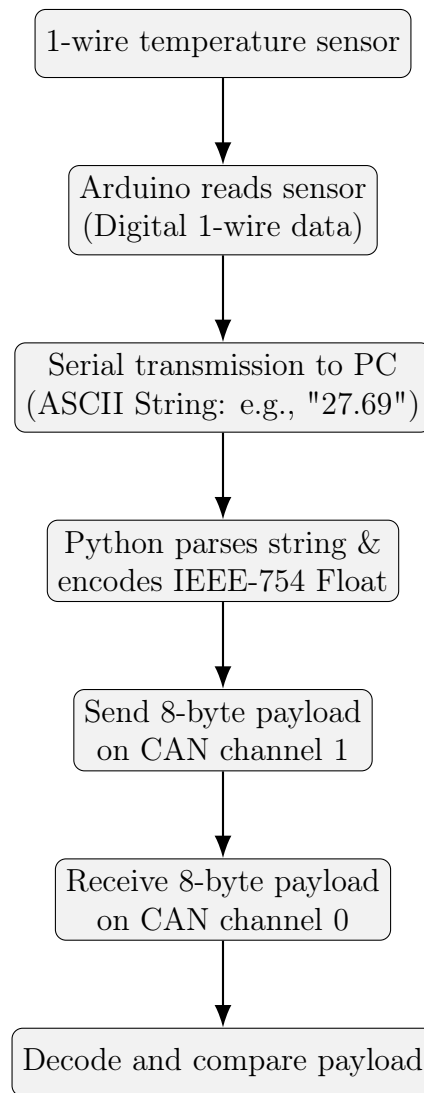


Figure 9: Data flow for sensor value transfer, detailing data type transitions.

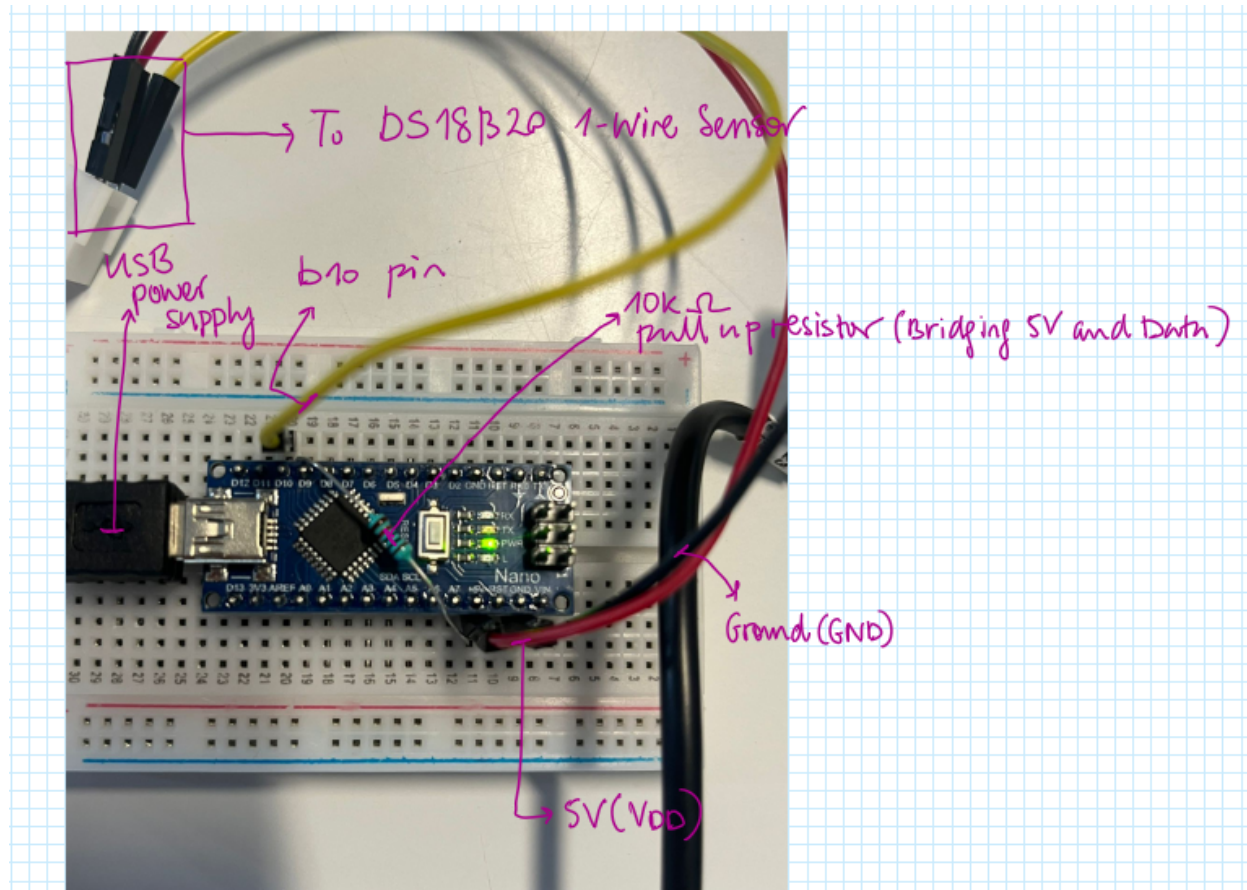


Figure 10: Sensor connection and powered Arduino

3.3 Payload Design

The final implementation used an 8-byte payload with an IEEE-754 single precision float for the temperature:

Table 3: CAN payload format used for Parts 3–5.

Byte(s)	Content	Meaning
0	Sequence number	Message counter, 0–255
1–4	Temperature float	IEEE-754 32-bit float, little-endian
5	Sensor ID	Sensor identifier, here 1
6	Status	0 = OK
7	Checksum	Sum of bytes 0–6 modulo 256

Important parameters from the real test:

- Serial port: COM4
- Serial baud rate: 9600 baud
- CAN bitrate: 500 kbit s⁻¹
- CAN ID for temperature frames: 0x123
- Send channel: 1
- Receive channel: 0
- Number of messages: 100

3.4 Collected Data

The successful run contains **100 messages** and all of them have status OK. The sent and received payloads were identical for every message.

Table 4: Part 3 successful transfer summary.

Quantity	Value
Messages planned	100
Messages received correctly	100
Payload mismatches	0
Minimum temperature	27.50 deg C
Maximum temperature	28.50 deg C
Mean temperature	27.93 deg C

```

1 Msg 001: SUCCESS | Seq: 000 | Sent: 27.69 °C | Received: 27.69 °C | Payload: 00 1F 85 DD 41
  01 00 C3
2 Msg 002: SUCCESS | Seq: 001 | Sent: 27.69 °C | Received: 27.69 °C | Payload: 01 1F 85 DD 41
  01 00 C4
3 Msg 003: SUCCESS | Seq: 002 | Sent: 27.69 °C | Received: 27.69 °C | Payload: 02 1F 85 DD 41
  01 00 C5
4 Msg 004: SUCCESS | Seq: 003 | Sent: 27.62 °C | Received: 27.62 °C | Payload: 03 C3 F5 DC 41
  01 00 D9
5 Msg 005: SUCCESS | Seq: 004 | Sent: 27.69 °C | Received: 27.69 °C | Payload: 04 1F 85 DD 41
  01 00 C7
6 Msg 006: SUCCESS | Seq: 005 | Sent: 27.62 °C | Received: 27.62 °C | Payload: 05 C3 F5 DC 41
  01 00 DB
7 Msg 007: SUCCESS | Seq: 006 | Sent: 27.62 °C | Received: 27.62 °C | Payload: 06 C3 F5 DC 41
  01 00 DC
8 Msg 008: SUCCESS | Seq: 007 | Sent: 27.62 °C | Received: 27.62 °C | Payload: 07 C3 F5 DC 41
  01 00 DD
9 Msg 009: SUCCESS | Seq: 008 | Sent: 27.62 °C | Received: 27.62 °C | Payload: 08 C3 F5 DC 41
  01 00 DE
10 ...
11 Msg 097: SUCCESS | Seq: 096 | Sent: 27.56 °C | Received: 27.56 °C | Payload: 60 E1 7A DC 41
  01 00 D9
12 Msg 098: SUCCESS | Seq: 097 | Sent: 27.56 °C | Received: 27.56 °C | Payload: 61 E1 7A DC 41
  01 00 DA
13 Msg 099: SUCCESS | Seq: 098 | Sent: 27.50 °C | Received: 27.50 °C | Payload: 62 00 00 DC 41
  01 00 80
14 Msg 100: SUCCESS | Seq: 099 | Sent: 27.56 °C | Received: 27.56 °C | Payload: 63 E1 7A DC 41
  01 00 DC

```

Listing 3: 100 Successful Temperature Readings

3.5 Analysis

The received payload was identical to the transmitted payload in all 100 messages. This proves that the sensor reading was correctly converted into the CAN payload and recovered after reception. The measured temperature varied between 27.50 deg C and 28.50 deg C, which is realistic for a room temperature sensor during the experiment.

4 Part 4: Automatic Error Checking

4.1 Task in Our Words

The fourth task automated the check from Part 3. The program should detect if a CAN message is missing, if the ID is wrong, if the DLC is wrong, if the payload is different, if the

sequence number is wrong, or if the checksum is invalid.

4.2 Automatic Checks Implemented

For every planned message the program checked:

1. Was a valid sensor value available?
2. Was a CAN frame with the expected ID received?
3. Was the DLC equal to 8?
4. Was the received payload identical to the sent payload?
5. Did the sequence number match?
6. Was the checksum valid?

4.3 Successful 100-message Test

The main automatic test produced 100 successful messages and no communication errors were observed in this sample.

Table 5: Automatic error checking result: successful run.

Result type	Count
Total messages	100
Successful messages	100
Errors total	0
Lost/timeout messages	0
Payload mismatches	0
Sequence errors	0
Checksum errors	0
Error rate	0.0%

The error rate is explicitly calculated as:

$$\text{error rate} = \frac{\text{number of errors}}{\text{number of planned messages}} \cdot 100\%$$

For the successful run:

$$\frac{0}{100} \cdot 100\% = 0.0\%$$

4.4 Additional Detection Test with Sensor Errors

An additional run was made where the measurement interval was shorter. This produced alternating valid and invalid sensor-read cycles. The automatic checker reported these as `SENSOR_ERROR`. This was not a CAN payload corruption; it was an acquisition problem before the CAN frame was created.

Table 6: Additional Part 4 run: sensor-read error detection.

Status	Count
SUCCESS	50
ERROR	50
Total planned messages	100
Detected error rate including sensor errors	50.0%

```

1 Msg 001: SUCCESS | Seq: 000 | Sent: 26.69 °C | Received: 26.69 °C | Payload: 00 1F 85 D5 41
  01 00 BB
2 Msg 002: ERROR | No valid temperature reading from sensor
3 Msg 003: SUCCESS | Seq: 002 | Sent: 26.69 °C | Received: 26.69 °C | Payload: 02 1F 85 D5 41
  01 00 BD
4 Msg 004: ERROR | No valid temperature reading from sensor
5 Msg 005: SUCCESS | Seq: 004 | Sent: 26.69 °C | Received: 26.69 °C | Payload: 04 1F 85 D5 41
  01 00 BF
6 Msg 006: ERROR | No valid temperature reading from sensor
7 Msg 007: SUCCESS | Seq: 006 | Sent: 26.69 °C | Received: 26.69 °C | Payload: 06 1F 85 D5 41
  01 00 C1
8 Msg 008: ERROR | No valid temperature reading from sensor
9 Msg 009: SUCCESS | Seq: 008 | Sent: 26.69 °C | Received: 26.69 °C | Payload: 08 1F 85 D5 41
  01 00 C3
10 Msg 010: ERROR | No valid temperature reading from sensor
11 ...
12 Msg 091: SUCCESS | Seq: 090 | Sent: 26.62 °C | Received: 26.62 °C | Payload: 5A C3 F5 D4 41
  01 00 28
13 Msg 092: ERROR | No valid temperature reading from sensor
14 Msg 093: SUCCESS | Seq: 092 | Sent: 26.62 °C | Received: 26.62 °C | Payload: 5C C3 F5 D4 41
  01 00 2A
15 Msg 094: ERROR | No valid temperature reading from sensor
16 Msg 095: SUCCESS | Seq: 094 | Sent: 26.56 °C | Received: 26.56 °C | Payload: 5E E1 7A D4 41
  01 00 CF
17 Msg 096: ERROR | No valid temperature reading from sensor
18 Msg 097: SUCCESS | Seq: 096 | Sent: 26.62 °C | Received: 26.62 °C | Payload: 60 C3 F5 D4 41
  01 00 2E
19 Msg 098: ERROR | No valid temperature reading from sensor
20 Msg 099: SUCCESS | Seq: 098 | Sent: 26.62 °C | Received: 26.62 °C | Payload: 62 C3 F5 D4 41
  01 00 30
21 Msg 100: ERROR | No valid temperature reading from sensor

```

Listing 4: Sensor data over CAN + Automatic Error Checking

4.5 Analysis

The automatic checker verified that all 100 CAN transmissions in the test sample were received correctly, confirming the 0.0% transmission error rate formula. However, because only 100 messages were tested, this does not mathematically prove that the real long-term CAN error probability is exactly zero. We can only state that no error was observed in this test and that the error probability is below the detection limit of this short measurement. With 100 messages and zero observed errors, a simple 95% upper confidence estimate is approximately 3%.

In the additional run, the checker also showed that the program can detect problems outside the CAN bus itself. This distinction is important: not every detected error is a CAN transmission error.

5 Part 5: Setup of a Large CAN Network

5.1 Task in Our Words

The final task transitioned the isolated interface onto a shared laboratory CAN bus. The objective was to participate in a multi-node environment, broadcasting our temperature sensor data using a designated unique CAN ID (0x105), while simultaneously monitoring the bus for foreign payloads. We were required to verify transmission integrity via an independent receiving node to prove that our frames successfully arbitrated and traversed the physical network.

5.2 Network Setup and Hardware Precautions

Transitioning to a shared bus requires critical physical layer adjustments. Before connecting, our local 120 Ω termination jumpers were explicitly deactivated. In a large bus topology, termination must only occur at the two physical extremes of the main trunk line. Leaving local terminators active on intermediate stubs would drop the parallel equivalent resistance of the bus well below 60 Ω , heavily loading the transceivers and corrupting the differential voltage levels.

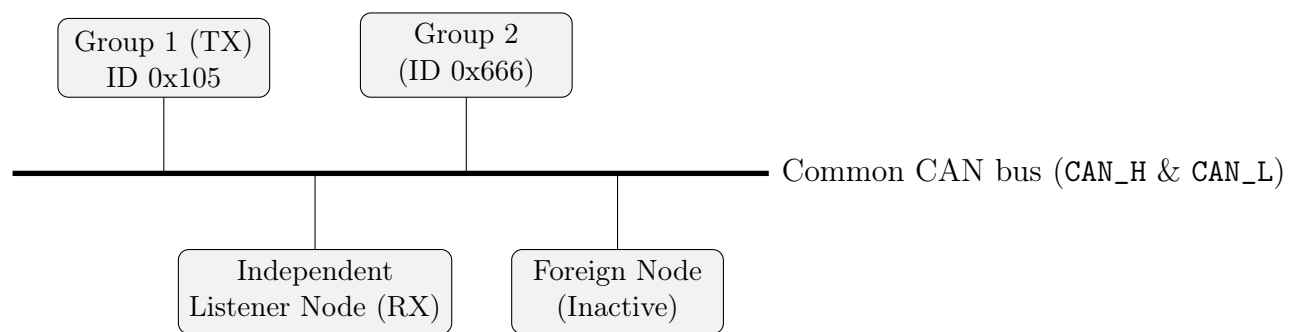


Figure 11: Topology of the shared CAN network. Local termination was disabled on our node, relying on the main trunk terminators.

5.3 Collected Data & External Receive Proof

During the logging period (approx. 290 seconds), our script successfully broadcasted 146 frames.

Table 7: Part 5 shared bus parameters and transmission summary.

Parameter	Value	Metric	Value
Shared bitrate	500 kbit/s	Total TX records logged	146
Assigned CAN ID	0x105	Payload Mismatches / Checksum Errors	0
Payload length	8 bytes	Min Temperature Transmitted	26.37 deg C
Sensor ID	1	Max Temperature Transmitted	26.50 deg C

```

In [4]: runfile('C:/Users/Student/Desktop/Lab_04_CAN_Listen.py', wdir='C:/Users/Student/Desktop')
[CanalystMessage ID=0x105 TS=0x14e2507 Data=001f85d3410100b9]
[CanalystMessage ID=0x105 TS=0x14e603e Data=011f85d3410100ba]
[CanalystMessage ID=0x105 TS=0x14eae5c Data=021f85d3410100bb]
[CanalystMessage ID=0x105 TS=0x14efc8f Data=031f85d3410100bc]
[CanalystMessage ID=0x105 TS=0x14f4b29 Data=041f85d3410100bd]
[CanalystMessage ID=0x105 TS=0x14f995a Data=05c3f5d2410100d1]
[CanalystMessage ID=0x105 TS=0x14fe78b Data=060000d44101001c]
[CanalystMessage ID=0x105 TS=0x15035fc Data=071f85d3410100c0]
[CanalystMessage ID=0x105 TS=0x150845e Data=081f85d3410100c1]
[CanalystMessage ID=0x105 TS=0x150d2bb Data=09c3f5d2410100d5]
[CanalystMessage ID=0x105 TS=0x151211e Data=0a1f85d3410100c3]
[CanalystMessage ID=0x105 TS=0x1516f38 Data=0b1f85d3410100c4]

```

(a) Independent bus monitor output showing ID 0x105.

Figure 12: Proof of Transmission: Photographs of a completely independent listener node successfully reading our payloads directly off the shared physical bus.

5.4 Analysis

The shared-bus test verified our node's ability to safely operate within a larger network topology. By successfully formatting the data frame with ID 0x105 and pushing it to the bus, the CAN controller automatically handled the physical arbitration phase.

During the laboratory session, another group successfully connected to the shared trunk using the identifier 0x666. While their presence physically confirmed the multi-node, multi-master nature of the bus, no photographic evidence was captured of their specific data frames arriving on our independent listener node. Therefore, to rigorously satisfy the empirical "Proof of Transmission" requirement, our analysis strictly focuses on the validated transmission of our own 0x105 frames. As shown in Figure 12, the external node successfully clocked in our frames, verified our DLC, and dumped the exact 8-byte payloads we transmitted. This irrefutably proves that our CAN transceiver successfully drove the differential bus voltages and that our data-link layer logic is fully operational.

6 Conclusion

This lab systematically validated the Controller Area Network (CAN) protocol from the physical layer up to the application layer. In Part 1, the USB-CAN hardware interface was successfully initialized, demonstrating the protocol's strict requirement for matching bitrates while allowing flexible identifier and DLC definitions. In Part 2, differential oscilloscope measurements visually confirmed the high noise immunity of the $V_{CAN_H} - V_{CAN_L}$ logic, while simultaneously demonstrating the hardware-level bit-stuffing required to prevent clock

drift during zero-payload transmissions.

In Parts 3 and 4, the application layer was developed to parse 1-wire digital sensor data into ASCII, convert it to an IEEE-754 float, and securely package it into an 8-byte CAN payload complete with sequence numbers and modulo-256 checksums. The automated test yielded a 0.0% transmission error rate across 100 frames, proving high localized reliability. Finally, in Part 5, the interface was transitioned to a shared bus topology. By properly disabling local terminators and broadcasting our designated `0x105` frames, we achieved verified transmission across the physical network, confirmed by external listener nodes. The results confirm a robust, fully functional implementation of a CAN sensor node.

A Key Source Code Extracts

The full Python scripts for this lab exceed several hundred lines. To highlight the core programmatic competencies required by the lab instructions, the most critical functional blocks from Parts 3, 4, and 5 are extracted below.

A.1 Data Payload Encoding and Decoding (Part 3)

This snippet from `lab4_part3_4_improved.py` demonstrates how the float temperature value from the Arduino is packed into a Little-Endian IEEE-754 byte array, combined with sequence and status flags, and secured with a modulo-256 checksum.

```
1 def checksum_8bit(payload_without_checksum):
2     """Sum of the first 7 payload bytes modulo 256."""
3     return sum(payload_without_checksum) & 0xFF
4
5 def make_payload(sequence_number, temperature_c):
6     seq = sequence_number & 0xFF
7     temp_bytes = struct.pack("<f", float(temperature_c)) # little-endian
8     # float
9     status = 0
10
11     first_7_bytes = (
12         seq,
13         temp_bytes[0], temp_bytes[1], temp_bytes[2], temp_bytes[3],
14         SENSOR_ID & 0xFF,
15         status & 0xFF
16     )
17     check = checksum_8bit(first_7_bytes)
18     return first_7_bytes + (check,)
19
20 def decode_payload(payload):
21     seq = payload[0]
22     temp_bytes = bytes(payload[1:5])
23     temperature_c = struct.unpack("<f", temp_bytes)[0]
24     sensor_id = payload[5]
25     status = payload[6]
26     received_checksum = payload[7]
27     calculated_checksum = checksum_8bit(payload[:7])
28     checksum_ok = (received_checksum == calculated_checksum)
29
30     return seq, temperature_c, sensor_id, status, checksum_ok
```

A.2 Automatic Error Verification Logic (Part 4)

This snippet highlights the sequential validation logic used to detect payload corruption, framing errors, and sequence drops.

```
1 # Exact CAN payload byte-for-byte check
2 if received_payload != sent_payload:
3     print(f"Msg {i + 1:03d}: ERROR | Payload mismatch")
4     error_count += 1
5     continue
6
7 # Decode and verify specific field content
8 seq, received_temp, received_sensor_id, status, checksum_ok =
9     decode_payload(received_payload)
```

```
9
10 if seq != (i & 0xFF):
11     print(f"Msg {i + 1:03d}: ERROR | Wrong sequence number")
12     error_count += 1
13 elif received_sensor_id != (SENSOR_ID & 0xFF):
14     print(f"Msg {i + 1:03d}: ERROR | Wrong sensor ID")
15     error_count += 1
16 elif status != 0:
17     print(f"Msg {i + 1:03d}: ERROR | Status byte is not OK")
18     error_count += 1
19 elif not checksum_ok:
20     print(f"Msg {i + 1:03d}: ERROR | Checksum error")
21     error_count += 1
22 else:
23     success_count += 1
```

A.3 Shared Bus Non-Blocking Loop (Part 5)

This snippet from `lab4_task5_fixed.py` shows the main loop architecture required to operate on a shared CAN bus. It continuously drains the receive buffer (to monitor foreign nodes) while periodically broadcasting our 0x105 sensor frames without using blocking sleep calls that would miss incoming traffic.

```
1 # Everyone on the shared bus must use the same bitrate.
2 SHARED_BITRATE = 500000
3 MY_GROUP_CAN_ID = 0x105
4
5 # ... (Setup omitted for brevity) ...
6
7 while True:
8     # 1) Always listen to the bus for incoming messages.
9     receive_and_print_bus_messages(dev, log_rows)
10
11     # 2) Send our own temperature periodically.
12     now = time.time()
13     if now >= next_send_time:
14         temp = get_temperature(ser, sequence)
15
16         if temp is not None:
17             send_temperature_message(dev, sequence, temp, log_rows)
18             sequence = (sequence + 1) & 0xFFFF
19         else:
20             print("[TX SKIP] No valid temperature reading.")
21
22     next_send_time = now + SEND_PERIOD_S
23
24     # Small delay to prevent 100% CPU utilization
25     time.sleep(0.01)
```